🚀 Инструкция по деплою Balloo Messenger

Версия: 1.0.0
Дата: 2024
Статус: ☑ Production Ready

📋 Содержание

1. Предварительные требования
2. Быстрый старт (Local Development)
3. Деплой на Vercel
4. Деплой на Railway
5. Деплой на собственном сервере
6. Конфигурация окружения
7. Проверка после деплоя
8. Решение проблем

Предварительные требования

☑ Обязательно

- **Node.js 18+** - Скачать
- **npm 9+** - Входит в Node.js
- **Git** - Для клонирования репозитория
- **Yandex OAuth приложение** - Создать

🔧 Опционально (для production)

- **PostgreSQL** - Для production БД (вместо SQLite)
- **Redis** - Для кэширования и rate limiting
- **SMTP сервер** - Для отправки email
- **Sentry** - Для мониторинга ошибок

Быстрый старт (Local Development)

1. Клонирование репозитория

```
git clone <repository-url>
cd messenger
```

2. Установка зависимостей

```
npm install
```

3. Настройка окружения

```
# Скопируйте пример .env
cp .env.example .env.local
```

Важные переменные для `.env.local`:

```
# JWT Secret (сгенерируйте свой!)
JWT_SECRET=your-32-character-secret-key-here

# Yandex OAuth
NEXT_PUBLIC_YANDEX_CLIENT_ID=your-client-id
YANDEX_CLIENT_SECRET=your-client-secret

# Database
DATABASE_URL="file:./prisma/dev.db"

# Application
NEXT_PUBLIC_APP_URL=http://localhost:3000
NODE_ENV=development
```

4. Инициализация базы данных

```
npm run db:setup
```

Это создаст:

- Базу данных `prisma/dev.db`
- Все необходимые таблицы
- Тестовые данные (если есть)

5. Запуск разработки

```
npm run dev
```

Откройте: **http://localhost:3000**

Деплой на Vercel

Почему Vercel?

- ☑ Бесплатный тариф
- ☑ Автоматический деплой из Git
- ☑ Встроенная CDN
- ☑ Автоматическое SSL
- ☑ Поддержка Next.js из коробки

1. Подготовка репозитория

```
git add .
git commit -m "Ready for production"
git push origin main
```

2. Создание проекта на Vercel

1. Зайдите на vercel.com
2. Нажмите **"Add New Project"**
3. Импортируйте Git репозиторий
4. Нажмите **"Import"**

3. Настройка переменных окружения

В Vercel Dashboard → Settings → Environment Variables:

Ключ	Значение
`NODE_ENV`	`production`
`JWT_SECRET`	`сгенерировать-32-символа`
`ENCRYPTION_KEY`	`сгенерировать-32-символа`
`NEXT_PUBLIC_APP_URL`	`https://your-app.vercel.app`
`NEXT_PUBLIC_YANDEX_CLIENT_ID`	`ваш-client-id`
`YANDEX_CLIENT_SECRET`	`ваш-client-secret`
`DATABASE_URL`	`postgresql://...` (для production)

4. Настройка базы данных

Вариант A: Vercel Postgres (рекомендуется)

1. В Vercel Dashboard → Storage → Create Database
2. Скопируйте `DATABASE_URL`
3. Добавьте в переменные окружения

Вариант B: Внешняя PostgreSQL

Используйте Neon, Supabase, или ваш собственный PostgreSQL.

5. Деплой

1. Нажмите **"Deploy"**
2. Подождите 2-5 минут
3. Готово! Ваша ссылка: `https://your-app.vercel.app`

6. Post-deployment

```
# Проверьте логи
vercel logs

# Пересоберите проект
vercel build
```

Деплой на Railway

Почему Railway?

- ☑ Простая настройка
- ☑ Автоматическое SSL
- ☑ Встроенная PostgreSQL
- ☑ Бесплатный тариф ($5 кредит)

1. Создание проекта

1. Зайдите на railway.app
2. Нажмите **"New Project"**
3. Выберите **"Deploy from GitHub repo"**

2. Настройка переменных

В Railway Dashboard → Variables:

```
NODE_ENV=production
JWT_SECRET=your-secret-key
DATABASE_URL=${{postgres.DATABASE_URL}}
```

3. Добавление PostgreSQL

1. Нажмите **"+ New"**
2. Выберите **"Database"** → **"PostgreSQL"**
3. Railway автоматически создаст и подключит БД

4. Деплой

1. Railway автоматически запустит деплой
2. Через 3-5 минут приложение готово
3. Ваша ссылка: `https://your-app.railway.app`

Деплой на собственном сервере

Требования

- Ubuntu/Debian 20.04+
- Node.js 18+
- Nginx (рекомендуется)
- PostgreSQL 14+

1. Подготовка сервера

```
# Обновите систему
sudo apt update && sudo apt upgrade -y

# Установите Node.js
curl -fsSL https://deb.nodesource.com/setup_18.x | sudo -E bash -
sudo apt install -y nodejs

# Установите PostgreSQL
sudo apt install -y postgresql postgresql-contrib

# Установите Nginx
sudo apt install -y nginx
```

2. Создание пользователя и директорий

```
# Создайте пользователя приложения
sudo useradd -m -s /bin/bash balloo
sudo su - balloo

# Перейдите в домашнюю директорию
cd ~

# Клонируйте репозиторий
git clone <repository-url> messenger
cd messenger
```

3. Установка зависимостей

```
npm install --production
```

4. Настройка окружения

```
cp .env.example .env.production.local
nano .env.production.local
```

Заполните переменные:

```
NODE_ENV=production
PORT=3000
HOSTNAME=0.0.0.0
DATABASE_URL="postgresql://balloo:password@localhost:5432/balloo"
JWT_SECRET=your-secure-secret-key
ENCRYPTION_KEY=your-32-char-key
```

5. Инициализация базы данных

```
# Создайте БД в PostgreSQL
sudo -u postgres psql
CREATE DATABASE balloo;
CREATE USER balloo WITH PASSWORD 'your_password';
GRANT ALL PRIVILEGES ON DATABASE balloo TO balloo;
\q

# Запустите миграции
npm run db:setup
```

6. Настройка PM2 (Process Manager)

```
# Установите PM2
sudo npm install -g pm2

# Запустите приложение
pm2 start npm --name "balloo" -- start

# Сохраните конфигурацию
pm2 save

# Настройте автозапуск при старте системы
sudo env PATH=$PATH:$(which node) pm2 startup systemd -u balloo --hp
/home/balloo
```

7. Настройка Nginx (Reverse Proxy)

```
sudo nano /etc/nginx/sites-available/balloo
```

Конфигурация:

```
server {
    listen 80;
    server_name your-domain.com;

    location / {
        proxy_pass http://localhost:3000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_cache_bypass $http_upgrade;
    }
}
```

Активируйте сайт:

```
sudo ln -s /etc/nginx/sites-available/balloo /etc/nginx/sites-enabled/
sudo nginx -t
sudo systemctl restart nginx
```

8. SSL сертификат (Let's Encrypt)

```
sudo apt install certbot python3-certbot-nginx
sudo certbot --nginx -d your-domain.com
```

Конфигурация окружения

🔑 Обязательные переменные

Ключ	Описание	Пример
`NODE_ENV`	Режим работы	`production`
`DATABASE_URL`	URL базы данных	`postgresql://user:pass@host:5432/db`

Ключ	Описание	Пример
`JWT_SECRET`	Секрет для JWT (32+ символов)	`x9Yz...`
`ENCRYPTION_KEY`	Ключ шифрования (32+ символов)	`a8Bc...`
`NEXT_PUBLIC_APP_URL`	URL приложения	`https://app.com`

🔐 Безопасность

Сгенерировать JWT_SECRET:

```
openssl rand -base64 32
```

Сгенерировать ENCRYPTION_KEY:

```
openssl rand -hex 32
```

📧 Email (опционально)

```
SMTP_HOST=smtp.yandex.ru
SMTP_PORT=587
SMTP_USER=your-email@yandex.ru
SMTP_PASS=your-app-password
```

🔔 Push-уведомления

```
# Генерация VAPID ключей
npx web-push generate-vapid-keys
```

```
NEXT_PUBLIC_VAPID_PUBLIC_KEY=ваш-public-key
VAPID_PRIVATE_KEY=ваш-private-key
VAPID_SUBJECT=mailto:admin@your-domain.com
```

🔑 Yandex OAuth

1. Зайдите в Yandex OAuth
2. Создайте новое приложение
3. Настройте redirect URI: `https://your-app.com/api/auth/yandex/callback`

4. Скопируйте Client ID и Client Secret

Проверка после деплоя

☑ Чек-лист

- ☐ Приложение доступно по URL
- ☐ Страница входа работает
- ☐ Регистрация пользователя работает
- ☐ Вход через Яндекс.ID работает
- ☐ Восстановление пароля работает
- ☐ База данных подключена
- ☐ Rate limiting работает
- ☐ CSRF защита работает
- ☐ HTTPS включён
- ☐ Логирование работает

🧪 Тестовые запросы

Проверка API:

```
# Проверка здоровья
curl https://your-app.com/api/health

# Проверка rate limiting
curl -X POST https://your-app.com/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"test@test.com","password":"test"}'
```

🔍 Мониторинг

Vercel:

```
vercel logs
```

PM2:

```
pm2 logs balloo
pm2 monit
```

Nginx:

```
sudo tail -f /var/log/nginx/access.log
sudo tail -f /var/log/nginx/error.log
```

Решение проблем

❌ Ошибка: "Database not found"

Решение:

```
npm run db:setup
```

❌ Ошибка: "JWT_SECRET not configured"

Решение:

```
# Добавьте в .env.local или переменные окружения
JWT_SECRET=your-32-character-secret-key
```

❌ Ошибка: "Yandex OAuth not working"

Проверьте:

1. `NEXT_PUBLIC_YANDEX_CLIENT_ID` установлен
2. `YANDEX_CLIENT_SECRET` установлен
3. Redirect URI совпадает: `https://your-app.com/api/auth/yandex/callback`

❌ Ошибка: "Rate limiting too strict"

Измените в `.env`:

```
RATE_LIMIT_MAX=100
RATE_LIMIT_WINDOW=60
```

❌ Ошибка: "Build failed"

Проверьте:

```
# Локальная сборка
npm run build

# Проверка типов
```

```
npx tsc --noEmit

# Проверка линтера
npm run lint
```

❌ Ошибка: "Memory limit exceeded"

Решение:

```
# Увеличьте лимит памяти Node.js
export NODE_OPTIONS="--max-old-space-size=4096"
```

📊 Масштабирование

Горизонтальное масштабирование

1. **PostgreSQL** - Используйте managed PostgreSQL (RDS, Cloud SQL)
2. **Redis** - Добавьте Redis для кэширования
3. **Load Balancer** - Используйте Nginx или HAProxy

Вертикальное масштабирование

- Увеличьте RAM сервера
- Увеличьте CPU
- Используйте SSD диски

🎯 Рекомендации для production

1. База данных

- Используйте **PostgreSQL** вместо SQLite
- Настройте **автоматические бэкапы**
- Используйте **connection pooling** (PgBouncer)

2. Кэширование

- Добавьте **Redis** для rate limiting
- Кэшируйте частые запросы
- Используйте CDN для статических файлов

3. Мониторинг

- **Sentry** - отслеживание ошибок
- **New Relic/Datadog** - мониторинг производительности
- **Uptime Robot** - мониторинг доступности

4. Безопасность

- Регулярно обновляйте зависимости
- Используйте **WAF** (Cloudflare)
- Настройте **CSP** (Content Security Policy)
- Регулярно делайте **бэкапы**

5. Логирование

- Используйте **ELK Stack** или **CloudWatch**
- Настройте алерты на критические ошибки
- Логируйте все действия пользователей

📞 Поддержка

При возникновении проблем:

1. Проверьте Solution Problems
2. Проверьте логи приложения
3. Проверьте переменные окружения
4. Создайте issue в репозитории

☑ Готово!

Ваше приложение готово к production! 🎉

Следующие шаги:

1. Настройте домен
2. Настройте SSL сертификат
3. Настройте мониторинг
4. Настройте бэкапы
5. Протестируйте все функции

Версия документа: 1.0.0
Последнее обновление: 2024
Статус: ☑ Готово к использованию